KANN AUDITS

Security Review
For IZKONEN

Prepared For: **IZKONEN**

Prepared By: **Kann, Volodymyr, cosminm53 ,Stranger8732**

Date Reported: **August 15, 2026**

Audit Summary

IZKONEN engaged Kann Audits to perform a security review of the IZK token and its emission system on Base. From August 12 to August 14, a team of security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

Protocol Overiew

IZKONEN is an ERC-20 token built on Base with an immutable core and a separate, replaceable emission module. The token has a fixed maximum supply of 38.69 trillion IZK, a yearly Fibonacci-based emission schedule running through 2048, and a one-time initial liquidity allocation. The protocol also includes a 3% max-wallet mechanism, emergency pausing, and governance-controlled roles for managing emissions and protocol configuration.

Scope

Repository: `https://github.com/marin-ctrl/izkonen-audit`

Audited Commit: `27bfe39`

Final Commit: `7f2137b`

Files:

- `src/IZKONEN.sol`
- `src/IZKONENMinter.sol`

Methodology

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms, and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

The auditing process specifically considered:

- Testing against common and uncommon attack vectors
- Ensuring compliance with industry best practices and known smart contract security standards
- Verifying contract logic matches the client's intended behavior
- Cross-referencing patterns with industry-standard implementations
- Manual line-by-line review by experienced auditors
- Access control verification
- Architecture and design analysis to ensure secure component interactions and clearly defined trust boundaries
- State transition and logic review to detect inconsistencies, unexpected behavior, or potential exploitation paths

Risk Classification

- **High** – result in directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.

- **Medium** – result in moderate loss of funds or impact multiple users, but only under specific conditions. They are exploitable without privileged access but typically require certain assumptions or edge cases.

- **Low/Info** – issues are non-exploitable findings that do not pose a direct security risk or impact the system's integrity. These issues are typically cosmetic, best-practice deviations, or related to documentation and compliance. While they do not require urgent remediation, they may be addressed to improve code quality and maintainability.

Findings Count

Critical/High	Medium	Low/Info
0	0	6

Findings Summary

ID	Title	Severity
L-01	Anti-whale limit computed from MAX_SUPPLY, not current supply	Low
I-01	Three tranche values deviate by one token from a strict Fibonacci series	Info
I-02	Replacement minter must preserve the current tranche index	Info
I-03	startTime_ Can Be Initialized Too Far in the Past	Info
I-04	initialLiquidityMinted is module-local, allowing repeated initial liquidity to break the final tranche	Info
I-05	Old and new minters can remain authorized at the same time	Info

Detailed Findings

L-01 - Anti-whale limit computed from MAX_SUPPLY, not current supply

Low

Description

`MAX_WALLET` is a constant equal to 3% of `MAX_SUPPLY`, or 1,160,700,000,000 IZK. The `_update` function enforces the limit after a transfer by checking whether `balanceOf(to)` exceeds `MAX_WALLET`. However, the public token documentation advertises the limit as "3% of current supply."

Because the early-year circulating supply is small relative to the fixed maximum-supply-derived limit, the concentration protection is effectively inactive until approximately 2041. During the early emission years, a single wallet can therefore hold a share of the circulating supply many times larger than the advertised 3%.

Impact

The advertised "maximum 3% per wallet" property is not enforced during the early years of the emission schedule. There is no direct loss of funds, but the implementation does not match the stated tokenomics and provides substantially weaker concentration protection than users may expect.

Recommendation

Calculate the wallet limit dynamically as 3% of `totalSupply()`

Remediation

FIXED

I-01 - Three tranche values deviate by one token from a strict Fibonacci series

Info

Description

The tranche comments and whitepaper language describe a Fibonacci-style emission schedule. The implemented whole-token tranche values mostly follow `tranche[n] = tranche[n - 1] + tranche[n - 2]`, but three entries differ from a strict recurrence by exactly one whole token.

The 2030 tranche is one token above the strict sum, the 2037 tranche is one token below the strict sum, and the 2043 tranche is one token above the strict sum. The discrepancy is economically immaterial at the scale of the schedule and appears to be a rounding or documentation-consistency issue rather than a technical defect. The total schedule plus initial liquidity still reaches the hard cap as tested.

Impact

The implementation does not exactly match a strict Fibonacci recurrence for every listed year. This may confuse reviewers or users comparing the implementation with the whitepaper, although the discrepancy is limited to one whole token in three tranches.

Recommendation

Update the whitepaper or code comments to state that the tranche table is Fibonacci-derived with three one-token rounding adjustments rather than a strict recurrence for every listed year.

Remediation

ACKNOWLEDGED

I-02 - Replacement minter must preserve the current tranche index

Info

Description

If the Minter contract is replaced, the new instance must be initialized with a `startingYearIndex_` that exactly matches the previous minter's `yearsMinted`. An incorrect or stale index can cause already-minted tranches to be repeated or future tranches to be skipped.

Impact

Incorrect migration configuration can disrupt the intended emission schedule and may result in extra tokens being minted or scheduled tranches being skipped.

Recommendation

During migration, verify that the replacement minter's `startingYearIndex_` exactly matches the current emission progress before granting it `MINTER_ROLE`.

Remediation

ACKNOWLEDGED

I-03 - startTime_ Can Be Initialized Too Far in the Past

Info

Description

The Minter can be initialized with a `startTime_` that is significantly earlier than the current block timestamp. Without a lower-bound validation, an incorrectly configured deployment can begin with an emission start time that is already far in the past, affecting the intended timing of the schedule.

Recommendation

For a first deployment, add a bounded validation such as:

```
require(
    startTime_ <= block.timestamp &&
    startTime_ >= block.timestamp - GRACE_PERIOD
);
```

For a replacement minter, derive `startTime_` directly from the retired minter's immutable `startTime` on-chain rather than accepting it as a new external input.

Remediation

FIXED

I-04 - initialLiquidityMinted is module-local, allowing repeated initial liquidity to break the final tranche

Info

Description

The `initialLiquidityMinted` flag is stored in the Minter contract rather than globally in the token. A replacement minter therefore starts with `initialLiquidityMinted = false`, allowing `mintInitialLiquidity()` to be executed again and mint an additional 38.5 billion IZK.

Impact

A replacement minter can mint the 38.5 billion IZK initial-liquidity allocation a second time, consuming supply reserved for later emissions. When the emission schedule reaches the final tranches, minting can revert because the hard cap has already been consumed.

Execution Cycle

Minter1 executes `mintInitialLiquidity()` legitimately → Minter1 is replaced by Minter2 with `initialLiquidityMinted = false` → Minter2 executes `mintInitialLiquidity()` again → the normal emission schedule continues → at index 22, `token.mint` reverts with `CapExceeded`.

Recommendation

Store the "initial liquidity already minted" state globally in the token, pass the flag explicitly into the replacement minter's constructor, or otherwise track the already-minted initial-liquidity allocation at the token level.

Remediation

ACKNOWLEDGED

I-05 - Old and new minters can remain authorized at the same time

Info

Description

The token does not maintain a single active minter. If a replacement minter is deployed with the correct `startingYearIndex_` but the previous minter is not deauthorized, both contracts can remain valid minters with the same emission progress. Since `mint()` is permissionless, each open emission window can be executed independently through both modules, causing the same tranche to be emitted twice.

Impact

Overlapping minters can cause double payouts to the configured market and treasury emission addresses, resulting in over-emission. The hard cap is then reached prematurely, preventing the largest late-stage tranches from being minted. The issue does not allow a third party to redirect the emission to an arbitrary address; the duplicated payouts still go to the configured emission recipients.

Execution Cycle

Minter A is operating → Minter B is added with the correct index → Minter A remains authorized → `mint()` is called on both A and B during each open window → each tranche is emitted twice → in a later year, `token.mint` reverts with `CapExceeded`.

Recommendation

Enforce a single active minter at the token level. For example, store an `activeMinter` address in the token and allow minting only from that address, with replacement performed atomically. Alternatively, track tranche execution directly in the token so the same tranche cannot be minted by more than one module.

Remediation

ACKNOWLEDGED

Disclaimers

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource, and expertise-bound effort in which we aim to identify as many issues as possible.

About Kann Audits

Kann Audits is a top-tier Web3 security audit company, trusted by leading projects and providing comphrensive audits and expert guidance to secure the most critical blockchain protocols.

Kann Audits provides services such as manual auditing, AI audits using the Kann AI Model, and incident response.

To learn more, visit `https://kannaudits.com`
To view our audit portfolio, visit `https://github.com/KannAudits`
To book an audit, message `https://t.me/KannAudits`